

19강

추상 클래스





➤ 추상 클래스란?(abstract class)

- 추상 클래스란 객체 생성은 불가능하지만, 상속을 통해 공통 기능과 미구현 메서드를 제공하는 클래스이다. 인스턴스 X

➤ 추상 클래스(abstract class) 의 특징

- 추상 메서드를 하나라도 가지면 반드시 추상 클래스여야 한다.

```
abstract class Calculator {
    abstract int add(int x, int y); { } ← X
}
```

- 추상 메서드는 구현 코드 없이 메서드의 선언만 있다.

예) `abstract int add(int x, int y);` // 선언만 있는 추상 메서드

`int add(int x, int y){ }` // { } 부분이 구현 내용임, 추상 메서드 아님

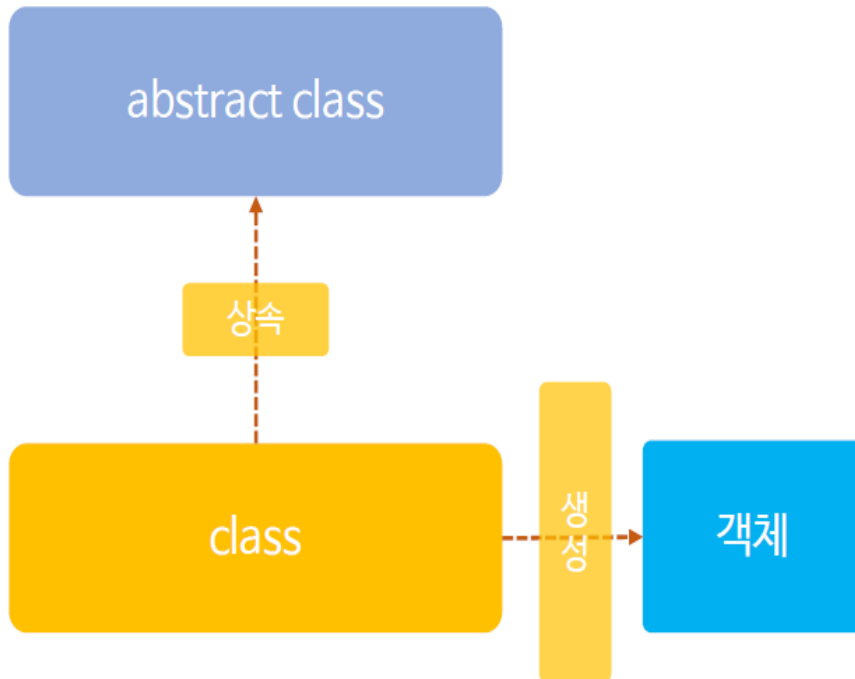
- Abstract** 예약어 사용
- 추상 클래스는 new(인스턴스 화) 할 수 없다

cf) concrete class (일반 클래스)

```
class Human extends Animal {
    void move() {
        System.out.println("걷는다");
    }
}
```



추상클래스는 A클래스, B클래스, C클래스들 간에 비슷한 필드와 메서드를 공통적으로 추출해 만들어진 클래스다. 클래스의 공통된 부분을 뽑아서 별도의 클래스(추상클래스)로 만들어 놓고, 이것을 상속해서 사용한다.



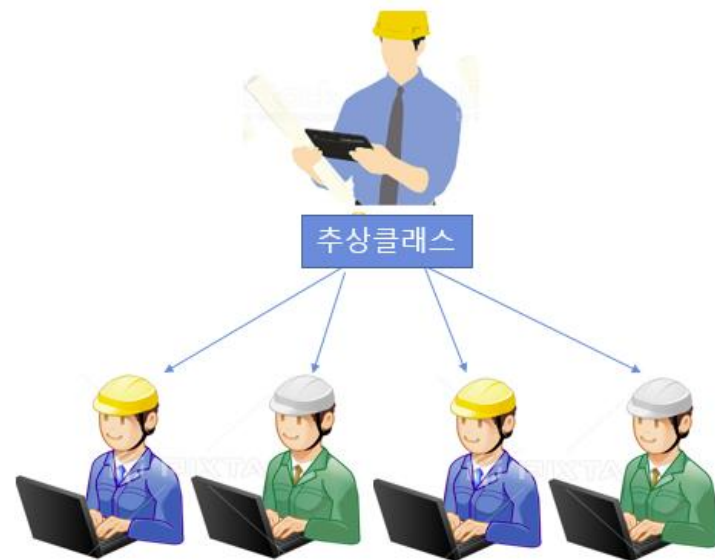
abstract 클래스의 특징

- 멤버변수를 가진다.
- abstract 클래스를 상속하기 위해서는 extends 를 이용한다.
- abstract 메서드를 가지며, 상속한 클래스에서 반듯이 구현해야 한다.
- 일반 메서드도 가질 수 있다.
- 일반 클래스와 마찬가지로 생성자도 있다.



➤ 추상 클래스 사용 이유

1. 공통된 멤버변수와 메서드를 통일할 목적
2. 실체클래스 구현시, 시간절약
3. 규격에 맞는 실체클래스 구현
4. 어떻게 구현해야 할지 구체적 방법을 모를 때, 각 하위 클래스에서 알아서 구현 해야 할 때



추상클래스를 상속받아 실체클래스를 구현하는 실무현장



➤ 추상 클래스 문법

클래스 앞에 **abstract** 키워드를 붙이면 추상클래스이다.

```
public abstract class 클래스명{  
    //멤버변수  
    //생성자  
    //메서드  
    //추상메서드  
}
```



➤ Computer 추상 클래스 구현

```
package abstractTest;

public abstract class Computer {

    // 추상(abstract) 클래스를 작성하는 이유 : 어떻게 구현해야 할지 구체적 방법을 모를 때
    //                                     각 하위 클래스에서 알아서 구현해야 할 때
    // public void display(); // 구현은 안하고 선언만 하겠다는 의미, 구현 코드가 없으면 추상메서드가 된다.
    // public void display(){ body} => body가 존재한다. 구현 코드가 없을 뿐이지 구현 부가 있다,
    // 하위 클래스가 각자 알아서 구현해야되는 경우는 추상(abstract)메서드로 작성한다.

    public abstract void display();
    public abstract void typing();

    // 모두 동적으로 구현해야되는 경우는 메서드로 작성한다.
    public void turnOn() {
        System.out.println("전원을 켭니다.");
    }
    public void turnOff() {
        System.out.println("전원을 끕니다.");
    }
}
```



- DeskTop 클래스는 Computer 추상 클래스를 상속받아 구현
- Notebook 클래스도 Computer 추상 클래스를 상속받아 구현
(단, abstract 키워드 직접 정의)

```
package abstractTest;

public class DeskTop extends Computer {

    @Override
    public void display() {
        System.out.println("DeskTop display()");
    }

    @Override
    public void typing() {
        System.out.println("DeskTop typing()");
    }

}
```

```
package abstractTest;

public abstract class Notebook extends Computer{

    @Override
    public void display() {
        System.out.println("NoteBook display()");
    }

}
```



- MyNoteBook 클래스는 NoteBook 클래스를 상속받아 구현
- ComputerTest 클래스에서 추상 클래스는 인스턴스 화 할 수 없음을 주의하

```
package abstractTest;

public class MyNoteBook extends NoteBook {

    @Override
    public void typing() {
        System.out.println("MyNoteBook typing()");
    }

}
```

```
package abstractTest;

public class ComputerTest {

    public static void main(String[] args) {

        // 추상 클래스는 인스턴스 화 할 수 없다.
        //Computer c1 = new Computer();
        Computer c2 = new DeskTop();
        //Computer c3 = new NoteBook();
        Computer c4 = new MyNoteBook();

        c2.display();
        c4.display();
    }
```

컴파일 실행



```
Console x
<terminated> ComputerTest [Java App
Desktop display()
NoteBook display()
|
```




➤ Final 예약어

- Final 변수는 값이 변경될 수 없는 상수이다.

public static final double PI = 3.14;

- Final 변수는 오직 한 번만 값을 할 당할 수 있다.
- Final 메서드는 하위 클래스에서 재정의(overriding) 할 수 없다.
- Final 클래스는 더 이상 상속되지 않는다.
- final 변수는 초기화 이후 값 변경이 불가능하다.
- 선언과 동시에 할당하거나, 생성자에서 한 번만 할당 가능하다

```
final int a;  
a = 10; // 가능 (첫 할당)  
a = 20; // ✕ 오류 (재할당 불가)
```



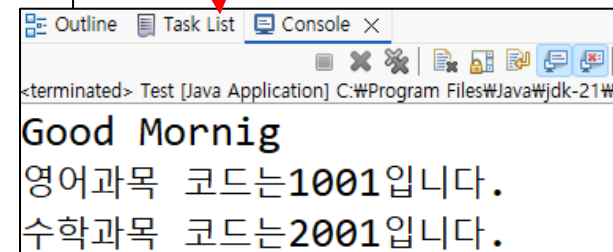
➤ 여러 자바 파일에서 공유하는 상수 값 정의 하기

```
package polymorphism;

class define{
    public static final int MIN = 1;
    public static final int MAX = 9999;
    public static final int ENG = 1001;
    public static final int MATH = 2001;
    public static final double PI = 3.14;
    public static final String GOOD_MORNIG = "Good Mornig";
}

public class Test {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println(define.GOOD_MORNIG);
        System.out.println("영어과목 코드는"+define.ENG+"입니다.");
        System.out.println("수학과목 코드는"+define.MATH+"입니다.");
    }
}
```



Static로 선언했으므로 인스턴스를 생성하지 않고 클래스 이름으로 참조가 가능하다.